

examples in which models lie in an attempt to deceive their users or third parties in scenarios simulating post-deployment situations. Like Goldowsky-Dill et al. (2025) we consider mitigating such threats with monitoring systems to detect when a model lies during deployment.

As the lies we expect in this use case are highly diverse, it may concern any object of belief and reason for lying. This use case has specific requirements stemming from the large amounts of messages that need to be monitored, for example a very low false positive rate for user-assistant chats Goldowsky-Dill et al. (2025), and low cost of running the detector.

A.4 MEASURING SUITABILITY FOR USE CASES

In this section, we motivate our choice of **balanced accuracy** as the primary evaluation metric of LIARS’ BENCH, building on insights into the requirements that lie detectors must satisfy across the use cases discussed above. Unlike recall, balanced accuracy ensures that performance is fairly assessed across lies *and* honest cases of our datasets. This is essential for the use cases *alignment evaluations*, where the reliability of a model’s self-reports is under scrutiny, and *safeguarding evaluations*, where detectors must identify strategic underperformance. Both applications may involve deliberately constructed dialogues around harmful topics, including deception. Methods that merely trigger on the *presence* of deception-related contexts, rather than on genuine lies (as observed by Goldowsky-Dill et al. (2025)) can appear strong under recall, but their high false-positive rates in such scenarios would undermine their value. Balanced accuracy penalizes such behavior, and therefore, provides a more faithful measure of detector reliability in these settings. Finally, the alpaca-based false-positive constraint operationalizes the requirement for large-scale monitoring that detectors must not flood oversight systems with false alerts during honest interactions. For a more thorough discussion of the three mentioned use cases, see Appendix A.

B LIMITATIONS IN OUR DEFINITION OF LYING: BELIEF AWARENESS

In Section 3, we define an AI assistant **lie** as a generated statement believed to be false by that same AI, with belief measured in separate, neutral contexts. This may differ from a more common-sense notion of lying in humans which requires belief awareness during the lie itself i.e., *knowingly* stating something believed to be false. Indeed, a theoretically perfect lie detector by our definition may flag genuine mistakes made by the assistant, for example due to confusing or misleading prompts.

This gap exists because we measure beliefs through consistent responses in neutral contexts, with no guarantee those same beliefs are salient during the generation of a contradiction with respect to them. We feel the development of tools and techniques for measuring this kind of belief awareness is an important research problem for the field, as the use of such methods could help bring a definition of lying in AI assistants closer to more intuitive definitions of lying in humans. Until such methods exist, we opt to use a potentially overly-permissive definition of lying to allow measurability and enable meaningful research progress.

C TAXONOMY OF LIE DETECTION METHODS

Lie detection methods are usually classified as *black-box* or *white-box*, depending on whether they have access to internals of the model under investigation Pacchiardi et al. (2023). We propose a more fine-grained taxonomy of lie detection methods based on the four criteria *model access*, *message scope*, *interactivity*, and *semantic verification*, as presented in Table 2.

Table 2: Taxonomy for lie detection methods

Property	Options and Descriptions
Model access	<i>Black-box</i> : has no access to model internals, such as its parameters or activations. <i>White-box</i> : can access model internals.
Message scope	<i>Assistant only</i> : only the model’s messages are available. <i>Dialogue</i> : messages by the model and the user are accessible. <i>Full</i> : can also see the system prompt.
Interactivity	<i>Static</i> : does not generate additional tokens. <i>Follow-up</i> : can use the model to generate additional tokens in the same conversation. <i>Unlimited</i> : can generate new messages, including across conversations.
Semantic verification	<i>Incapable</i> : can not semantically verify any claims. <i>Capable</i> : can verify claims, e.g. through access to the internet, language models, or humans.

We illustrate this taxonomy using two example methods. A linear probe that predicts lying from the activations of all tokens in the conversation operates with *white-box* model access, a *full* message scope, is *incapable* of semantic verification, and exhibits *static* interactivity. A second method that presents an independent language model with all user and assistant messages and asks it whether the original model lied is a *black-box* approach with *dialogue* message scope, is *capable* of semantic verification, and also exhibits *static* interactivity.

This taxonomy helps to understand which methods are suitable to detect certain types of lies.

For example, consider a model being evaluated for its alignment by being asked whether it has access to dangerous types of knowledge, or whether it would ever create copies of itself without authorization. Here, the object of belief is context-independent self-knowledge, which makes it difficult to verify semantically, and if the statement is just a simple “Yes” or “No”, the tokens alone carry insufficient information to determine its honesty. As a result, a detector with *black-box* model access and *static* interactivity cannot reliably assess the statement’s honesty, as it lacks access to a channel with the necessary information. Our experimental results with the *LLM-as-a-Judge* method on the *harm-pressure-knowledge-report* model organism, which have these properties, support this claim.

Similarly, the same type of detector would fail when future models make claims about the real-world too complex to fact-check. Such situations can be simulated by defining the object of belief in the system prompt and constraining the message scope to exclude it. This, however, is difficult to achieve for activation-based detectors, as the activations they access may contain information from all previous tokens.